

State-Space Models

Replacing attention with a running memory · video 2:23

This lesson answers one question: can a language model work without attention? **State-space models** keep a fixed-size memory, updated once per token: $h \leftarrow a \cdot h + (1 - a) \cdot u$. With a fixed decay a , a tiny model matches attention (1.648 vs 1.644); with a **selective** decay chosen by each token (the idea behind Mamba) it does better on these short texts (1.591). The big win is at inference: the state never grows, and each new token costs the same 6 μ s whether the context is a thousand tokens or a hundred thousand.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. A running memory	▶ 0:08-0:49	2	☐
2. The experiment	▶ 0:49-1:24	2	☐
3. Inference: memory and time	▶ 1:24-2:15	2	☐

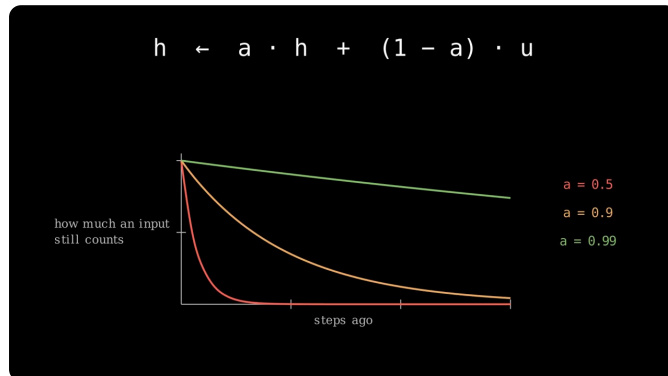
Before you start: Deep Dive episodes 10-11 (FlashAttention, sliding windows) and LLMs in Practice episode 2 (the KV cache). Code: `code/d29_state_space` (PyTorch only; about 10 minutes on a CPU).

CONCEPT 1 OF 3

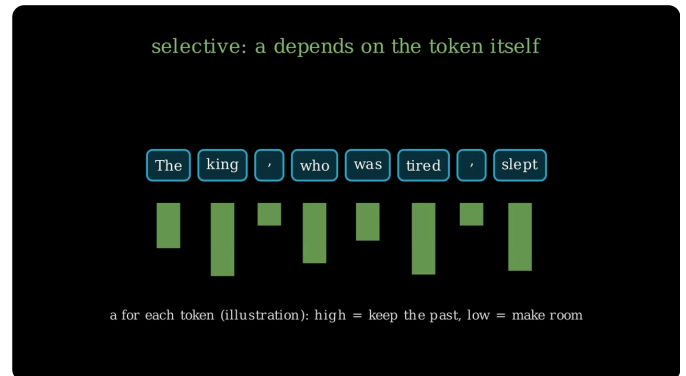
A running memory

▶ Video 0:08–0:49

Attention compares every new token with every earlier one and keeps a cache that grows with the text. **State-space models** keep a fixed-size memory updated once per token. The simplest version, for every channel: $\mathbf{h} \leftarrow \mathbf{a} \cdot \mathbf{h} + (\mathbf{1} - \mathbf{a}) \cdot \mathbf{u}$, a running, decaying average of the inputs $\mathbf{u}$. With a fixed $\mathbf{a}$ per channel, some channels remember for a long time ($\mathbf{a} = 0.99$) and others only a few steps ($\mathbf{a} = 0.5$).



▶ 0:35 $\mathbf{h} \leftarrow \mathbf{a} \cdot \mathbf{h} + (\mathbf{1} - \mathbf{a}) \cdot \mathbf{u}$: an input s steps ago still counts $\mathbf{a}^s$; $\mathbf{a} = 0.5$ forgets fast, 0.99 remembers long.



▶ 0:47 Selective: $\mathbf{a}$ depends on each token, so the model chooses what to keep (values illustrative).

The key idea behind Mamba-style models is to make $\mathbf{a}$ depend on the token itself: $\mathbf{a}_t = \text{sigmoid}(\mathbf{W} \mathbf{x}_t)$. The model can then choose, at every step, what to keep and what to forget: a **selective** state space.

KEY IDEA

A recurrence carries the past in a fixed-size state; making its decay input-dependent lets it decide what is worth keeping.

Check 1: A running memory

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

With a fixed $\mathbf{a} = 0.9$, how much does an input from 10 steps ago still count, relative to when it arrived ($\mathbf{a}^{10}$)?

1.2 CALCULATION

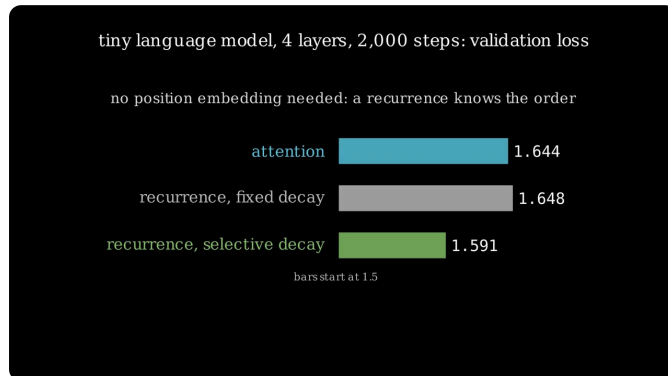
After how many steps has a channel with $\mathbf{a} = 0.99$ forgotten half of an input?

CONCEPT 2 OF 3

The experiment

▶ Video 0:49–1:24

The tiny model's attention is replaced by each recurrence (no position embedding is needed: a recurrence knows the order by itself) and trained 2,000 steps. Validation loss: attention **1.644**, fixed decay **1.648** (about the same), selective decay **1.591**, better than attention here.



▶ 1:07 Validation loss: attention 1.644, fixed decay 1.648, selective decay 1.591.

here = 64-character texts: nearby characters matter most
a fixed-size state must squeeze the whole past
exact recall from far back: where attention keeps its edge

▶ 1:22 On 64-character texts nearby characters matter most; exact recall from far back is where attention keeps its edge.

“Here” means 64-character texts, where nearby characters matter most. A fixed-size state must squeeze the whole past into a few numbers, and research has found that exact recall from far back is where attention keeps its edge (compare the copy task of episode 11).

KEY IDEA

Recurrents can match or beat attention on local patterns; long, exact lookup is their weak spot.

Check 2: The experiment

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 MULTIPLE CHOICE

Which model reached the lowest validation loss in the episode?

- A. Attention
- B. Recurrence, fixed decay
- C. Recurrence, selective decay
- D. All equal

2.2 SHORT ANSWER

Why does the recurrent model not need a position embedding?

.....

CONCEPT 3 OF 3

Inference: memory and time

▶ Video 1:24-2:15

The big win is at inference. Per layer (128 numbers per token, float32), attention's cache grows with every token: **0.98 MiB** after a thousand tokens, **97.66 MiB** after a hundred thousand. The recurrent state stays at **0.50 KiB**, forever.

Time to process one new token in one layer, on this CPU: attention takes 37.9 μs with a thousand tokens of context, 312.6 μs with ten thousand, 11,334 μs with a hundred thousand; the recurrence about **6.4 μs** every time. So some recent models mix the two: AI21's Jamba interleaves Mamba layers with a few attention layers, cheap memory for most of the work and exact lookup where it's needed.

memory per layer while generating (128 numbers per token, float32)

after	attention KV cache	recurrent state
1,000 tokens	0.98 MiB	0.50 KiB
100,000 tokens	97.66 MiB	0.50 KiB

time to process one new token in one layer (this CPU)

context	attention	recurrence
1,000 tokens	37.9 μs	6.3 μs
10,000 tokens	312.6 μs	6.4 μs
100,000 tokens	11,334 μs	6.4 μs

▶ 1:36 Per layer: attention's KV cache 0.98 MiB after 1,000 tokens, 97.66 MiB after 100,000; the recurrent state stays 0.50 KiB.

▶ 1:54 One new token, one layer: attention 37.9 μs → 11,334 μs as the context grows; the recurrence 6.3–6.4 μs .

KEY IDEA

Constant memory and constant time per token, at the price of compressing the past.

Check 3: Inference: memory and time

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

How large would one layer's KV cache be after 1,000,000 tokens (128 numbers, float32, keys and values)?

3.2 HANDS-ON CODE

Try it yourself. Run the recurrence by hand on a constant input $u = 1$ with $a = 0.5$, for 5 steps from $h = 0$.

```
h = 0.0
for _ in range(5):
    h = 0.5 * h + 0.5 * 1.0
    print(h)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. A running memory · rewatch ▶ 0:08-0:49

1.1 About 0.35. $0.9^{10} \approx 0.349$.

1.2 About 69. $0.99^s = 0.5 \rightarrow s = \ln 0.5 / \ln 0.99 \approx 69$.

2. The experiment · rewatch ▶ 0:49-1:24

2.1 C. 1.591 vs 1.644 and 1.648.

2.2 It processes tokens one after another, so the order is built into how the state is updated; attention, by itself, is blind to order (episode 4). Each update depends on the previous state, so swapping tokens changes the result.

3. Inference: memory and time · rewatch ▶ 1:24-2:15

3.1 About 977 MiB. $2 \times 1,000,000 \times 128 \times 4 \text{ bytes} \approx 1.02 \times 10^9 \text{ bytes}$.

3.2 0.5, 0.75, 0.875, 0.9375, 0.96875. The state approaches the input, closing half the gap each step.